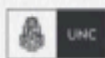


CERTIFICACIÓN UNIVERSITARIA



FCEFYN



Manual para el
alumno

Seguridad Back End



WE WANT
SKILLS!

mundosE

Manual para el alumno: Seguridad Back End.....	2
Objetivo del encuentro:.....	2
Contenido a desarrollar:.....	2
1. Marco conceptual de seguridad de Back End.....	3
2. Vulnerabilidades típicas e impacto.....	4
3. Protección de bases de datos.....	6
4. Prevención de inyecciones SQL y validación de entradas.....	7
5. OWASP Dependency-Check (SCA).....	8
Bibliografía.....	10

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este encontrarás un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el alumno: Seguridad Back End

Objetivo del encuentro:

Comprender y aplicar prácticas de seguridad del lado del servidor (Back End) para proteger APIs, bases de datos y componentes críticos de la aplicación. Reconocer vulnerabilidades típicas, prevenir inyecciones SQL, diseñar controles de acceso robustos y utilizar herramientas de apoyo (SCA con OWASP Dependency-Check y DAST con Burp Suite Community Edition) dentro de un proceso de desarrollo seguro.

Contenido a desarrollar:

- ✓ Marco conceptual de seguridad de Back End
- ✓ Vulnerabilidades típicas e impacto en Back End
- ✓ Protección de bases de datos
- ✓ Prevención de inyecciones SQL y validación de entradas
- ✓ OWASP Dependency-Check (SCA)
- ✓ Burp Suite Community Edition (DAST)

1. Marco conceptual de seguridad de Back End

La seguridad de Back End abarca decisiones de diseño, controles técnicos y hábitos de ingeniería que resguardan la lógica de negocio y los datos cuando se procesan del lado del servidor. En términos prácticos, es el conjunto de prácticas que evita que un error puntual (por ejemplo, un *endpoint* que no valida correctamente, o una cuenta de base de datos con privilegios excesivos) se convierta en una brecha mayor con impacto operativo, legal y reputacional.

Para organizar el estudio, es útil pensar la seguridad en capas:

- (a) exposición (red, TLS, *headers*);
- (b) aplicación (*framework*, controladores, validación, manejo de errores);
- (c) datos (consultas, ORM, cifrado, permisos, auditoría); y,
- (d) gestión de secretos y configuración (KMS/Vault, rotación, segregación por entorno).

La primera idea clave es **confianza mínima**: el Back End nunca debe asumir que la entrada viene “limpia”. Aunque el Front End valide, el servidor tiene que validar otra vez según un contrato explícito (tipos, rangos, formatos) y, sobre todo, decidir cómo falla: respuestas claras (códigos 400/422 cuando corresponda), sin filtrar detalles internos (nombres de tablas, trazas), y con registro interno suficiente para diagnosticar sin exponer PII. La segunda idea es **control de acceso en el servidor**: la autorización se verifica en cada *endpoint*, para cada operación y sobre cada recurso, independientemente de la UI. La tercera es **mínimo privilegio**: cuentas de servicio y usuarios con exactamente los permisos necesarios, ni uno más; y separación de responsabilidades (por ejemplo, lectura y escritura con credenciales distintas cuando aplica).

Otro pilar es la **observabilidad orientada a seguridad**. No es suficiente responder 200/400/500: el Back End debería registrar de forma estructurada intentos de acceso, errores de validación, fallas de autenticación y eventos de autorización (permitidos/denegados), aplicando *scrubbing* para que los *logs* no contengan contraseñas, *tokens* completos o documentos de identidad. Con métricas (latencia, tasas de 401/403/5xx) y trazas distribuidas, es posible detectar patrones anómalos (picos fuera de horario, inputs sospechosos repetidos, *endpoints* golpeados en secuencia) y activar respuestas.

En la práctica, la seguridad se “fabrica” durante todo el ciclo de vida. Un enfoque **shift-left** integra controles en el *pipeline*: análisis de composición de *software* (SCA) para conocer vulnerabilidades en dependencias; pruebas de seguridad

automáticas donde sea viable; y listas de verificación en revisiones de código (validación en servidor, parametrización de consultas, manejo de errores neutro, límites y paginación por defecto). La **evidencia** de estos controles (reportes, artefactos firmados, SBOM, resultados de pruebas) permite decisiones informadas frente a hallazgos y acelera auditorías internas o externas.

Un error frecuente es confundir “cumplimiento” con “seguridad real”. Un *checklist* es un medio, no un fin. La seguridad de Back End se prueba en ejecución: ¿qué pasa si elimino validaciones en el cliente?, ¿si repito una solicitud con un token caducado?, ¿si fuerzo un parámetro con valores extremos o inesperados? Al incorporar una cultura de pruebas que incluyan estos escenarios, se fortalece la resiliencia del sistema.

Finalmente, el **contexto** importa. No todas las aplicaciones tienen el mismo riesgo: un servicio que maneja datos de salud o financieros demanda exigencias mayores que un blog institucional. Aun así, hay mínimos no negociables que aplican en todos los casos: transporte cifrado actual (TLS), secretos fuera del repositorio y con rotación, consultas parametrizadas por defecto, validación/normalización consistente, control de acceso en el servidor y logs sin PII crítica. Estos mínimos forman tu “línea de base” para evaluar la madurez de un Back End.

Qué deberías lograr al finalizar esta sección: poder explicar por qué la seguridad se diseña en capas, describir el principio de mínimo privilegio con ejemplos concretos (por ejemplo, qué permisos debe tener la cuenta de servicio de una app) y enumerar los mínimos no negociables que vas a verificar siempre.

2. Vulnerabilidades típicas e impacto

Las vulnerabilidades más frecuentes en Back End suelen repetirse por atajos o supuestos peligrosos. **Inyección SQL (SQLi)** aparece cuando se construyen consultas concatenando cadenas con entradas del usuario. La diferencia conceptual es que el input no solo aporta un valor, sino que altera la estructura de la sentencia (por ejemplo, cerrando una comilla y agregando **OR 1=1**). Las variantes incluyen *error-based* y *time-based/blind*. Señales de alerta: respuestas incoherentes, tiempos de respuesta diferentes ante *payloads* con operadores, y mensajes de error que delatan tablas o columnas. Impacto: lectura masiva, modificación o destrucción de datos y, en algunos motores, funciones con efectos en el sistema operativo.

Autenticación y gestión de sesión inseguras abarcan prácticas como guardar contraseñas sin *hashing* fuerte (bcrypt/Argon2 son recomendables), *tokens* de larga vida sin rotación/revocación, JWT sin expiración o firmados con algoritmos

inseguros, y sesiones que no se renuevan al elevar privilegios. Un flujo de login que revela si un usuario existe también incrementa la superficie de ataque (enumeración). El impacto típico es secuestro de cuentas y persistencia de accesos indebidos; se agrava si los *tokens* otorgan *scopes* amplios o si la *app* no invalida sesiones cerradas.

En **exposición de datos sensibles**, los problemas más comunes son logs con PII o secretos, *endpoints* de depuración visibles, *stack traces* completos devueltos al cliente y transporte sin TLS moderno. Estas fugas no solo dañan la reputación: alimentan ataques posteriores (*credential stuffing*, ingeniería social) y pueden implicar sanciones regulatorias. Para mitigarlo: mensajes de error neutros, *scrubbing/masking* en *logs*, deshabilitar depuración en producción y revisar que TLS esté correctamente configurado.

El **control de acceso roto** ocurre cuando la autorización no se verifica de forma sistemática en el servidor, o se asume que el cliente “no va a intentar” ciertas operaciones. Es habitual que rutas secundarias (exportaciones, métricas, búsquedas masivas) carezcan de controles, o que IDs de recursos sean predecibles sin validar que el sujeto esté autorizado. Impacto: lectura/escritura indebida, escalamiento horizontal (acceso a datos de otros usuarios) y, en condiciones extremas, escalamiento vertical.

Para desarrollar intuición práctica, reproduce fallas en un entorno de laboratorio. Prueba un *endpoint* de búsqueda con inputs extremos (muy largos, comillas, comentarios) y observa cómo cambian las respuestas con y sin parametrización. Revisa con un proxy interceptante cómo viaja un *token* (cabeceras, *cookies*), si se caduca y renueva correctamente, y si *endpoints* “secundarios” repiten controles de autorización. Además, usa métricas: picos de 401/403 pueden indicar fuerza bruta; crecimientos de 5xx, errores de manejo que quizá estén filtrando datos.

El impacto de estas vulnerabilidades va más allá del aspecto técnico. Una exfiltración de datos puede derivar en notificaciones obligatorias a usuarios y reguladores, multas, investigación forense, costos de remediación y pérdida de confianza. Por eso es clave priorizar remediaciones por riesgo: ¿qué es explotable ahora?, ¿qué datos quedan expuestos?, ¿qué controles limitan el daño si el primer anillo falla?

Qué deberías lograr al finalizar esta sección: identificar señales de alerta de SQLi, listar prácticas seguras de autenticación/sesión, reconocer ejemplos de exposición de datos y describir cómo verificar la autorización en *endpoints* menos evidentes.

3. Protección de bases de datos

Proteger la base de datos requiere combinar cifrado, control de acceso y observabilidad. En **tránsito**, activa TLS entre la aplicación y el motor: versiones actuales, suites recomendadas, verificación de certificados y políticas consistentes en todas las conexiones (no solo en producción). En **reposo**, aplica cifrado a nivel volumen/archivo y, cuando sea necesario, cifrado granular de columnas (por ejemplo, documentos de identidad, tarjetas, emails). Recuerda que el cifrado en reposo **no reemplaza** controles de aplicación ni limita accesos privilegiados: es un anillo adicional para reducir impacto ante pérdida de soportes o acceso físico.

El **principio de mínimo privilegio** es fundamental. Define cuentas de servicio por aplicación o por microservicio, con permisos acotados a las operaciones y esquemas necesarios. Evita ejecutar la app con usuarios de administración. Cuando exista separación lectura/escritura, usa credenciales distintas. En motores que lo soporten, aplica **Row-Level Security (RLS)** o políticas por columna para ajustar la visibilidad según rol o atributo. Mantén catálogos de quién puede hacer qué, y revísalos periódicamente para evitar la “*creep*” de privilegios con el paso del tiempo.

La **gestión de secretos** debería estar a cargo de un servicio dedicado (por ejemplo, Vault o gestores nativos de nube). Evita variables de entorno planas en hosts compartidos y, sobre todo, no incluyas claves ni contraseñas en repositorios (aunque sean privados). Diseña la **rotación** para que sea posible con mínima fricción, idealmente sin redeploy —o con automatización que reduzca tiempos de exposición— y limita el alcance de cada secreto (*scopes* y TTL).

La **auditoría y monitoreo** te dan visibilidad para reaccionar. Configura logs de consultas relevantes (con cuidado de no volcar PII en claro), alertas por operaciones anómalas (SELECT masivos fuera de horario, orígenes inusuales) y control de comandos peligrosos (DROP/ALTER). Usa almacenamiento de logs seguro y con retención definida por riesgo y normativa. Un procedimiento de **revisión periódica** debería comprobar que los permisos efectivos de las cuentas de servicio coinciden con la política declarada.

No descuides **backups y entornos**. Las copias deben estar cifradas, con acceso restringido y pruebas regulares de restauración (para validar tanto el dato como la disponibilidad de claves). Muchos incidentes se agravan por dumps antiguos sin cifrar en ubicaciones públicas o en entornos de desarrollo. Para esos entornos, aplica anonimización/pseudonimización. Si mantienes réplicas o lagos de datos para analítica, replica las políticas de seguridad: el eslabón más débil suele ser una copia secundaria mal configurada.

Por último, integra estos controles en tu flujo de trabajo: tests que fallen si una conexión no usa TLS, scripts que verifiquen permisos (por ejemplo, que la cuenta

de la *app* no tenga **DROP**), y listas de verificación en despliegues. La protección de datos no es un “*set-and-forget*”: cambian versiones, librerías, equipos y requisitos regulatorios; por eso necesitas procesos repetibles que confirmen la postura de seguridad en el tiempo.

Qué deberías lograr al finalizar esta sección: ser capaz de proponer un perfil de permisos para una *app* concreta, describir cómo y por qué cifrarías una columna sensible, y explicar un plan rápido de acción si se filtra una credencial de base de datos.

4. Prevención de inyecciones SQL y validación de entradas

La defensa más efectiva contra SQLi es **parametrizar siempre**. Los *prepared statements* separan los datos de la estructura de la sentencia y evitan que operadores especiales del input alteren el SQL. Aunque uses un ORM o un *query builder*, debes elegir los métodos que aceptan parámetros y no interpolar *strings*. Para consultas dinámicas (ordenamiento, filtros), define **listas blancas** de columnas y operadores permitidos, y convierte la preferencia del usuario en una estructura controlada que tu capa de datos traduzca a SQL seguro. Establece **paginación y límites** por defecto para reducir el impacto de consultas costosas.

La **validación y normalización** complementan a la parametrización. Valida tipos (número, fecha, UUID), rangos y formatos con reglas claras; impón longitudes máximas razonables y rechaza inputs que no cumplan el contrato. Para dominios finitos, usa enumeraciones o listas blancas. Diferencia entre **validar** (aceptar lo que cumple) y “sanear” (intentar corregir lo incorrecto); la primera es preferible porque reduce ambigüedades. En APIs que reciben JSON complejos, apóyate en **esquemas** (por ejemplo, JSON Schema) para automatizar validaciones y generar errores consistentes.

El **manejo de errores** es parte de la seguridad. Hacia el cliente, usa mensajes neutros y códigos apropiados; reserva el detalle técnico para logs internos, aplicando *scrubbing/masking*. Evita exponer *stack traces* o nombres de tablas/columnas. Suma **controles operativos**: *rate limiting* y *timeouts* para frenar exploración automatizada de *payloads*, límites de tamaño de solicitud y monitoreo de tasas de 4xx/5xx por endpoint. En APIs públicas, combina con mecanismos anti-abuso y análisis de patrones.

Convierte estas prácticas en hábitos verificables. En pruebas unitarias y de contrato, incluye **entradas hostiles** (comillas, comentarios, valores fuera de rango, Unicode inesperado) y verifica que la respuesta sea controlada y que los *logs* no filtren PII. En revisiones de código, busca señales de riesgo: concatenaciones en consultas, construcción manual de **ORDER BY/WHERE**, validaciones hechas

únicamente en el cliente, funciones “helper” que interpelen SQL sin parámetros. Documenta una **checklist** de Back End y haz que sea parte de la definición de “hecho” (DoD) del equipo.

Un ejemplo práctico: un *endpoint* `/items?sort=precio&dir=desc&query=teléfono` puede implementarse con un mapeo interno de `sort` (por ejemplo, `precio`, `nombre`, `fecha_alta`) hacia identificadores de columnas validados; `dir` restringido a `asc/desc`; y `query` transmitido como parámetro de búsqueda al motor (idealmente con capacidades nativas de full-text o vistas/materializadas revisadas). En ese diseño, ningún valor del usuario se interpola directamente en la sentencia, y ante valores inválidos se responde con 400/422.

Qué deberías lograr al finalizar esta sección: saber escribir consultas parametrizadas, diseñar filtros dinámicos seguros con listas blancas, y construir un plan de validación/normalización y manejo de errores coherente para una API real.

5. OWASP Dependency-Check (SCA)

OWASP Dependency-Check es una herramienta de **análisis de composición de software (SCA)**: inspecciona las dependencias (directas y transitivas) de un proyecto y las contrasta con bases de vulnerabilidades para informar **riesgos por versión**. Su utilidad no es “arreglar” vulnerabilidades, sino brindar **visibilidad** para decidir: actualizar, mitigar (configurar), reemplazar o —de forma excepcional y documentada— aceptar temporalmente un riesgo mientras se planifica la remediación. Está disponible para múltiples ecosistemas (por ejemplo, Maven/Gradle en Java, Node.js, .NET) y genera reportes útiles (HTML/JSON/JUnit) que puedes publicar como artefactos y usar como gates en CI/CD.

Para empezar, ejecuta Dependency-Check en tu entorno local o intégralo en tu *build*. Examina el reporte HTML: por cada hallazgo, lee la descripción, la severidad y la versión afectada. Presta atención a posibles **falsos positivos** (emparejamientos ambiguos por CPE), y afina la configuración cuando sea necesario. Es buena práctica definir **umbrales** (por ejemplo, fallar la *build* si hay vulnerabilidades críticas) y documentar **excepciones** con caducidad, obligando a una revisión futura. Integrar SCA en el pipeline te permite detectar riesgos antes de que una versión avance hacia testing o producción.

SCA se potencia al combinarlo con un **SBOM** por *release* (un inventario formal de componentes y versiones). Contar con SBOM acelera la respuesta ante nuevas CVEs: sabes con precisión qué versiones usan tus artefactos y dónde desplegaron. Además, habilita conversaciones claras con seguridad y cumplimiento (licencias, políticas de actualización, ventanas de mantenimiento). En equipos grandes,

publica los reportes junto al artefacto (por ejemplo, como parte del paquete en el registro) y registra las decisiones (actualizar, cambiar librería, mitigar) para generar trazabilidad.

Un flujo de adopción razonable es: (1) ejecutar SCA en cada *merge* a ramas principales; (2) fijar umbrales de severidad y reglas de bloqueo; (3) generar y adjuntar reportes al artefacto; (4) mantener un *backlog* visible de hallazgos con responsables y fechas; y (5) realizar “sprints de limpieza” periódicos para reducir deuda. Recuerda que actualizar dependencias es una inversión en seguridad y mantenibilidad: postergarlo hace más costosa la remediación futura.

Para práctica individual, elige un proyecto propio, corre Dependency-Check y clasifica hallazgos en tres categorías: (a) actualización inmediata (parches disponibles y bajo riesgo de roturas); (b) mitigación por configuración (activar una opción segura o deshabilitar una función peligrosa); y (c) reemplazo o rediseño (cuando la librería quedó abandonada o su riesgo es estructural). Documenta la decisión y, si hay excepciones, fíjales una fecha de vencimiento y criterio de cierre.

Qué deberías lograr al finalizar esta sección: interpretar un reporte SCA, distinguir falsos positivos de hallazgos reales, proponer acciones de remediación y definir umbrales de severidad razonables para tu pipeline.

Recursos para profundizar

- OWASP Dependency-Check — Proyecto oficial: <https://owasp.org/www-project-dependency-check/>
- GitHub Dependency-Check (documentación y releases): <https://github.com/dependency-check/DependencyCheck>
- Burp Suite — Documentación oficial: <https://portswigger.net/burp/documentation>
- Burp Suite Community Edition — Descarga: <https://portswigger.net/burp/communitydownload>
- OWASP Cheat Sheet Series — SQL Injection Prevention: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP Top 10 (Web Application Security Risks): <https://owasp.org/www-project-top-ten/>
- OWASP ASVS — Estándar de verificación: <https://owasp.org/www-project-application-security-verification-standard/>
- OWASP DevGuide — ASVS (introducción para desarrolladores): <https://devguide.owasp.org/en/06-verification/01-guides/03-asvs/>
- JSON Schema — Sitio oficial: <https://json-schema.org/>
- PostgreSQL — SSL/TLS en clientes (ejemplo de DB): <https://www.postgresql.org/docs/current/libpq-ssl.html>
- NIST — Secure Software Development Framework (SSDF): <https://csrc.nist.gov/projects/ssdf>

Bibliografía

- NIST. (2022). Secure Software Development Framework (SP 800-218).
<https://csrc.nist.gov/projects/ssdf>
- OWASP. (s. f.). Cheat Sheet Series. <https://cheatsheetseries.owasp.org/>
- OWASP. (s. f.). Top 10 Web Application Security Risks.
<https://owasp.org/www-project-top-ten/>
- OWASP. (s. f.). Application Security Verification Standard (ASVS).
<https://owasp.org/www-project-application-security-verification-standard/>
- OWASP Foundation. (s. f.). Dependency-Check.
<https://owasp.org/www-project-dependency-check/>
- PortSwigger. (s. f.). Burp Suite — Documentation.
<https://portswigger.net/burp/documentation>